

State

Let an object change its behavior when its internal state changes — so it appears to change its class at runtime.

DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

DIVE 02 · 1 km
Train window
Theses, examples, terms

→

DIVE 03 · 100 m
Park walk
How the pieces relate

→

DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Turns a tangled state machine into one object per state — each state owns its own behavior and decides the next move.

THE CONTOURS

• Behavior follows state

— the same call acts differently per mode

• One class per state

replaces sprawling enum + switch ladders

• States drive transitions

— each one knows which mode comes next

USE IT WHEN — GoF applicability

Behavior depends on state

and a single object must act differently across many modes

Big conditionals on a state field

sprawl across methods and beg to be split per state

Behavioral family: Strategy · State · Observer · Command · Iterator — State looks like Strategy but the states swap themselves, not the client.

↓ DIVE DEEPER

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Context

holds the current ConcreteState; delegates each request to it

State

the interface declaring behavior shared by every state

ConcreteState

implements one mode's behavior; may trigger the next transition

IN THE WILD

XState / statecharts

finite states & transitions as first-class data

TCP/IP state machine

LISTEN → ESTABLISHED → CLOSED per RFC 793

Spring Statemachine

states, events & guarded transitions for Java

React useReducer

next state computed from current state + action

Context → delegates request() to its current State → state runs, then may call context.transitionTo(next).

GoF intent: “Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.”

↓ DIVE DEEPER

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Event arrives

request() on Context

→

Delegate to state

Context → current State

→

State decides

act, then pick next mode

→

✓ transitionTo(next)

behavior visibly changes

WHY IT FOLLOWS

→ Per-state class ⇒ each mode's behavior is local, named and testable

→ Delegation ⇒ Context stays thin; it just forwards to the live state

→ State-owned transitions ⇒ the next mode lives beside the logic that earns it

➤ Often a Flyweight when states are stateless · Context may set its first state via a Factory · structurally identical to Strategy, but State swaps itself while Strategy is chosen by the client.

STRUCTURE · UML

Context

```
classDiagram
    class Context {
        - state: State
        + request(): void
        + transitionTo(s: State): void
    }
    class State {
    }
    class ConcreteState {
    }
    Context --> State
    Context --> ConcreteState
    State <|-- ConcreteState
```

Context forwards request() to its current State; a ConcreteState may call transitionTo() to swap the active mode.

WHAT IT BUYS YOU

✓ Localizes behavior — all logic for one mode sits in a single, named class

✓ Makes transitions explicit — switching state is a deliberate object swap, not a flag flip

✓ Removes big conditionals — no state-checking if/switch scattered through methods

✓ States can be shared — stateless state objects can be reused as flyweight singletons

✓ More classes — many small classes — overkill for two or three trivial modes

↓ DIVE DEEPER

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Allow an object to alter its behavior when its internal state changes; the object will appear to change its class.”

— GoF, “Design Patterns”, 1994

THE CODE

```
interface State { insertCoin(m: Machine): void; }
class NoCoin implements State {
  insertCoin(m: Machine) {
    m.setState(new HasCoin()); // self-transition
  }
}
class Machine {
  private state: State = new NoCoin();
  setState(s: State) { this.state = s; }
  insertCoin() { this.state.insertCoin(this); }
}
```

VARIANTS

Context-owned

Context decides each transition; states stay pure behavior

State-owned

each ConcreteState calls transitionTo() — classic GoF

Table-driven FSM

(state, event) → next state held in a transition table

Enum + switch

small FSM as one enum and a switch — no class explosion

Statecharts

hierarchical & parallel states with guards (XState / SCXML)

Stateless singletons

share one immutable instance per state as a flyweight

COMPARE THE ALTERNATIVES

Property	State	Strategy	enum+switch	FSM lib
Swaps own transition	✓	✗	~	✓
Client picks behavior	✗	✓	✓	~
One class per mode	✓	✓	✗	✗
Scales to many states	~	✓	✗	✓
Visualizable FSM	~	✗	✗	✓

Best fit → State: self-driven modes · Strategy: client-chosen algorithm · enum+switch: tiny FSM · FSM lib: complex statecharts. (“~” = partial.)

INTERVIEW PITFALLS

Q “State vs Strategy?”

Same structure; State objects swap themselves as conditions change, a Strategy is chosen once by the client and doesn't self-transition.

Q “Who triggers transitions?”

Either the ConcreteState (state-owned) or the Context (context-owned) — a deliberate trade between coupling and centralization.

Q “Why not just a switch?”

For a few modes a switch is fine; State pays off when conditionals sprawl across methods and modes keep growing.

Q “Can states be shared?”

Yes — if a state keeps no per-context data, reuse one immutable instance as a flyweight singleton.

NUANCES & GOTCHAS

• Who owns transitions — state-owned couples states to each other; context-owned centralizes the map but fattens Context

• Share stateless states — if a state holds no per-context data, reuse one instance as a singleton/flyweight

• State explosion — modes × sub-modes multiply classes fast; statecharts or a table tame it

• Entry / exit actions — run setup/teardown on switch; add guards so transitions can be conditional

• vs Strategy — same UML, opposite control: State transitions itself, Strategy is set by the client

• Logic is distributed — behavior spread across classes can hurt the at-a-glance view of the whole machine

WATCH OUT

Anti-pattern: Two or three trivial modes rarely justify a class per state — a single enum field with a small switch is clearer and lighter than the ceremony.

Modern take: Reach for a statechart library (XState, Spring Statemachine) or a reducer once states, events and guards grow — you get visualizable, testable transitions for free.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994)

v1